

Weekly Progress Report - Week 6

Improve performance of existing online trackers for small object detection [UAV Videos]

Group: The Convolutioners

{Aagam Sheth, Mahima Parekh, Khevan Thanki, Riddhi Bhargava}

Reporting Period: March 30 – April 4, 2026

Submission Date: April 4, 2026

Outline of Performed Tasks:

- Designed and implemented a custom BoT-SORT tracker integrating the optimised SAHI + YOLO11 detection pipeline with the Siamese Re-ID module
- Applied Camera Motion Compensation (CMC) via sparse optical flow to correct UAV ego-motion between frames
- Evaluated the full custom tracker pipeline across all 7 VisDrone2019-MOT validation sequences on GPU
- Implemented Phase 1 and Phase 2 of a structured four-phase hyperparameter sweep for tracker improvement
- Logged and ranked all hyperparameter trials by a composite score weighted across IDF1, HOTA, and MOTA
- Analysed per-sequence performance to identify failure modes and areas for further improvement

Custom BoT-SORT Tracker Design

This week’s primary contribution was the full integration of all developed components into a unified tracking pipeline. The custom tracker combines:

- **Detection front-end:** YOLO11 fine-tuned on VisDrone, with optional SAHI tiling
- **Motion prediction:** Constant-velocity Kalman filter for forward-projecting track bounding boxes
- **Appearance Re-ID:** The optimised Siamese network (ResNet18, $d = 64$, $\tau = 1.0$)
- **Two-stage association:** Hungarian algorithm applied over a blended IoU and cosine appearance cost

The two-stage association operates as follows. In Stage 1, high-confidence detections are matched against all active tracks using a blended cost:

$$\text{cost} = (1 - \alpha) \cdot d_{\text{IoU}} + \alpha \cdot d_{\text{appearance}}, \quad \alpha = 0.65 \quad (1)$$

In Stage 2, remaining unmatched tracks are associated with low-confidence detections using IoU cost only, since low-confidence detections are insufficiently reliable to use appearance cues.

Stale tracks are pruned and unmatched detections satisfying birth thresholds are initialised as new tracks.

Each detection patch is resized to 128×128 and passed through the Siamese ResNet18 branch to produce a 128-dimensional L2-normalised embedding. Track embeddings are maintained via an exponential moving average across frames to account for gradual appearance change.

Camera Motion Compensation

A key challenge in UAV tracking is ego-motion: even a slow drone drift causes all predicted track positions to shift simultaneously, artificially inflating IoU distances and producing spurious mismatches independent of any actual object movement. To address this, Camera Motion Compensation (CMC) was implemented using sparse optical flow between consecutive frames. The estimated affine transformation is used to warp existing track bounding box predictions into the current frame's coordinate system before both the IoU and appearance matching stages are applied.

On a single-sequence diagnostic, CMC reduced identity switches from 67 to 64. A minor increase in track fragmentation was observed, attributed to the fact that Phase 1–2 association thresholds were calibrated prior to CMC integration; a re-sweep with CMC enabled is expected to recover this and yield further benefit.

Full Benchmark Evaluation

To assess the end-to-end performance of the custom tracker, two complete GPU benchmark runs were executed across all 7 VisDrone2019-MOT validation sequences. The two pipeline variants evaluated are:

- **YOLO+Siamese+BoT-SORT**: Plain YOLO11 detector with no tiling, paired with the custom Siamese tracker
- **SAHI+Siamese+BoT-SORT**: SAHI tiling enabled on the same YOLO11 detector, paired with the custom Siamese tracker

Each pipeline is assessed on standard MOT metrics including MOTA, MOTP, IDF1, HOTA, DetA, and AssA, as well as raw counts of false positives, missed detections, and identity switches. The comparison aims to quantify the trade-off between the improved small object recall offered by SAHI tiling and its associated increase in false positives, and to assess how each pipeline compares against the preliminary CPU baseline established in earlier weeks. Runtime in frames per second (FPS) is also recorded to assess practical deployability.

Structured Tracker Hyperparameter Sweep (Phases 1–2)

To systematically improve tracker performance beyond the default configuration, a structured four-phase hyperparameter sweep was implemented. All trials are logged to `hparam_trials.csv` for full reproducibility, and candidates are ranked by a composite score that weights IDF1, HOTA, and MOTA to prioritise identity consistency:

$$\text{score} = 0.45 \cdot \text{IDF1} + 0.35 \cdot \text{HOTA} + 0.20 \cdot \text{MOTA} \quad (2)$$

Phase 1 focused on association gate parameters, sweeping `appearance_weight`, `match_cost_thresh`, `min_iou_gate`, and `assoc_iou_thresh` over 10 trials. The goal was to identify how much weight should be given to appearance versus spatial overlap during the matching stage, and how strictly the association gates should be set.

Phase 2 focused on track lifecycle parameters, sweeping `min_hits`, `max_age`, and `new_track_thresh` over 24 trials. The aim was to determine how quickly new tracks should be confirmed and how long lost tracks should be retained, which is particularly important in UAV footage where small objects appear and disappear rapidly.

Phases 3 and 4, covering Kalman filter dynamics and local re-association parameters respectively, have not yet been implemented and remain the primary avenue for further tracker improvement. These phases are planned for completion in the final week alongside result compilation and report preparation.

Tentative List of Tasks for Next Week:

1. Compile all experimental results across all pipeline configurations into a unified comparison
2. Perform detailed qualitative and quantitative analysis of tracking successes and failure modes
3. Prepare the final project report covering all phases: dataset preparation, detection, Re-ID, and tracking
4. Prepare presentation slides summarising methodology, key results, and insights